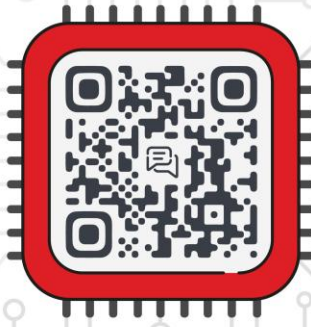




نظري

7



22



2640 sp

كلية الهندسة المعلوماتية

السنة الثالثة

Advanced Pipelining

د. سيرا أستور



RB Informatics; 24/11/2025

بنیان الحواسيب 2

تذكرة :

Latency هي الزمن اللازم لتفادي ال hazard بين تعليمتين موجودتين مرتبطتين بين بعضهما البعض

Basic Pipeline Scheduling and Loop Unrolling

أفضل أداء لتقنية Pipelining يتحقق عندما يكون خط الأنابيب ممتلئاً بالعمليات باستمرار دون توقف. لتحقيق ذلك، يجب ترتيب التعليمات بحيث يتم تنفيذ تعليمات غير مرتبطة معاً في نفس الوقت داخل مراحل مختلفة من خط الأنابيب.

أما التعليمات المعتمدة على نتائج تعليمات سابقة، فيجب تأخير تنفيذها بعدد من دورات الساعة (latency) لتعليمية المصدر، حتى يصبح الناتج جاهزاً للاستخدام.

Example: Adding a Scalar to a Vector

في هذا المثال، يتم إضافة قيمة ثابتة s إلى كل عنصر في المتجه x :

$for (i = 999; i \geq 0; i = i - 1)$

$x[i] = x[i] + s;$

كل تكرار من الحلقة مستقل عن الآخر، لذلك يمكن تنفيذ عدة تكرارات بشكل متداخل باستخدام pipeline scheduling أو loop unrolling، مما يؤدي إلى تحسين الأداء.



Code in MIPS

Loop: L.D F0 ,o(R1)	# F0= array element
ADD.D F4, F0 ,F2	# add scalar in F2
S.D F4,o(R1)	# store result
DADDUI R1 ,R1,-8	# decrement pointer;8 bytes (per DW)
BNE R1 ,R2,Loop	# branch R1!=R2

نفترض أن السجل R1 يحتوي في البداية على عنوان العنصر الأول من المصفوفة (العنوان الكامل) ← (R1) 0، وأن السجل F2 يحتوي على القيمة الثابتة s.

يتم تنفيذ الحلقة باستخدام تعليمات MIPS كما يلي:

تعليمة L.D تقوم بتحميل عنصر من المصفوفة إلى السجل العائم F0 .
 بعد ذلك، تعليمة ADD.D تضيف القيمة الثابتة s الموجودة في F2 إلى العنصر المحمل.
 ثم تعليمة S.D تخزن النتيجة مرة أخرى في نفس موقع الذاكرة.
 بعد معالجة العنصر، يتم إنقاص المؤشر R1 بمقدار 8 بايت للانتقال إلى العنصر السابق في المصفوفة، لأن كل عنصر من نوع Double Word.
 أخيراً، تعليمة BNE تتحقق مما إذا كان R1 لا يساوي R2، وفي هذه الحالة تستمر الحلقة بالتنفيذ.

ملاحظة :

-نستخدم مسجلات ال float مثل f0 ولكن العناوين نستخدم R1,R2 لأن العناوين لا يمكن إلا أن تكون integer
 -العنوان الكامل لازم يكون integer و موجب (الذاكرة لا تحوي عنوان سالب)
 ال offset ممكن يكون موجب و سالب



Latencies of FP Operations Used

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

FP :Floating point

يوضح هذا الجدول (Latency) لعمليات الفاصلة العائمة، أي عدد دورات الساعة التي يجب الانتظار بها بين تعليمتين لتجنب حدوث stall في خط الأنابيب

- العمود الأول يبين تعليمة تنتج قيمة، التعليمة التي تليها تستهلك هذه القيمة (مثل ضع ناتج الجمع في a وال a تكون مدخل لتعليمة تالية تستهلكها)
- والعمود الثاني يبين نوع التعليمة التي تستخدم هذه النتيجة
- أما العمود الثالث فيمثل عدد دورات الساعة الفاصلة المطلوبة بينهما
-

على سبيل المثال :

إذا كانت التعليمة الأولى هي عملية FP ALU op (التعليمات الحسابية و المنطقية) متبوعة بعملية FP ALU op مرتبطة بها تماما لأنها تستهلكها وأيضا تعليمة حسابية تحتاج latency 3 دورات ساعة.

- عملية FP ALU قبل تعليمة تخزين Store double تحتاج إلى دورتين.
- تعليمة Load double قبل عملية FP ALU تحتاج إلى دورة واحدة فقط.
- أما Load double متبوعة ب Store double فلا تحتاج إلى أي تأخير.

ملاحظة :

هذه الارتباطات موأى load مع أي store
ال load لقيمة نفس القيمة عم استعمالها بال store للارتباط عملنا هذول القيم

تذكرة : للتخلص من ال hazard يوجد لدينا طريقتين :

الأولى : نجلب تعليمات غير مرتبطة ونضعها خلف بعض .

الثانية : حتى ولو نفذنا الطريقة الأولى يوجد تعليمات مرتبطة، فنبعد التعليمات المرتبطة عن بعضها (نفكهم) حتى لا يصبح تضارب فبالتالي تكون ال latency أديش بدك تفك بيناتهم

Without Any Scheduling (Ignoring Delayed Branch for Now)

	Clock cycle issued
Loop: L.D F0,0(R1)	1
stall	2
ADD.D F4,F0,F2	3
stall	4
stall	5
S.D F4,0(R1)	6
DADDUI R1,R1,#-8	7 # Results are in exe stage
stall	8
BNE R1,R2,Loop	9 # Comparison is in ID stage

Unscheduled code requires 9 clock cycles

بناء على الجدول السابق قمنا بوضع عدد ال stall

في الدورة الأولى يتم تنفيذ تعليمة Load لتحميل العنصر إلى السجل F0 . بعدها تحدث حالة توقف لأن تعليمة ADD.D تعتمد على ناتج التحميل، مما يفرض الانتظار دورة واحدة قبل تنفيذها. بعد تنفيذ ADD.D، نحتاج إلى دورتين إضافيتين قبل تنفيذ تعليمة Store لأن نتيجة الجمع غير جاهزة مباشرة. ثم يتم تحديث المؤشر R1، ولكن تعليمة الفرع BNE تعتمد على النتيجة، مما يؤدي إلى توقف إضافي.

بسبب هذه التوقيفات، تحتاج الحلقة غير المجدولة إلى 9 دورات ساعة لإتمام تكرار واحد. اعتبرنا كل تعليمة تحتاج نبضة واحدة لأن ال cpi بال pipeline واحد

Basic Scheduling

Basic Scheduling

```

Loop:  L.D  F0,0(R1)      1
        DADDUI R1,R1,#-8  2
        ADD.D F4,F0,F2    3
        stall             4
        stall             5
        S.D  F4,8(R1)     6
        BNE R1,R2,Loop    7

```

The clock cycles are reduced to 7!! But only three instructions are actually doing the work inside the loop!!

❧ في هذا المثال :

تم تطبيق جدولة أساسية للتعليمات بهدف تقليل ال stalls (عدد حالات التوقف في خط الأنابيب).

تم نقل تعليمة تحديث المؤشر DADDUI مباشرة بعد تعليمة Load، لأنها مستقلة ولا تعتمد على ناتج التحميل. بهذا الشكل يتم استغلال دورات الانتظار بدل ترك خط الأنابيب فارغاً.

بعد ذلك تُنفَّذ تعليمة ADD.D، تليها دورتا انتظار latency، ثم يتم تنفيذ تعليمة Store وأخيراً تعليمة الفرع BNE.

نتيجة هذا الترتيب، انخفض عدد دورات الساعة من 9 إلى 7 دورات لكل تكرار.

ومع ذلك، نلاحظ أن ثلاث تعليمات فقط هي التي تقوم بالعمل الفعلي داخل الحلقة Load، Add، Store، بينما باقي التعليمات تُستخدم للتحكم وتحديث المؤشر

$S.D\ F4,8(R1)$

نلاحظ أننا غيرنا العنوان كان لدينا بالسابق :

$$(-8 + 8 = 0)$$

$S.D\ F4,0(R1)$

Loop Example with Delayed Branch Slot

- One delayed branch slot
 - Unscheduled code: **10 clock cycles**
- | | | |
|----|--------------------|----------------------|
| 1 | Loop: L.D F0,o(R1) | |
| 2 | stall | #load interlock |
| 3 | ADD.D F4,F0,F2 | |
| 4 | stall | #data hazard (F4) |
| 5 | stall | #data hazard (F4) |
| 6 | S.D F4,o(R1) | |
| 7 | DADDUI R1,R1,#-8 | |
| 8 | stall | #branch interlock |
| 9 | BNE R1,R2,Loop | |
| 10 | stall | #delayed branch slot |

نلاحظ أننا فقط قمنا بزيادة ال stall الأخيرة باللون الأزرق

في هذا المثال نأخذ بعين الاعتبار وجود Delayed Branch Slot واحد، أي أن التعليمة التي تأتي بعد الفرع تُنفَّذ دائماً قبل الانتقال.

عند تنفيذ الكود بدون أي scheduling ، تظهر عدة حالات توقف داخل خط الأنابيب.

بعد تعليمة Load يحدث توقف قبل تنفيذ تعليمة الجمع.

كذلك، بعد تعليمة ADD.D تظهر حالاً توقف بسبب اعتماد تعليمة Store على الناتج F4.

عند الوصول إلى تعليمة الفرع BNE، يحدث توقف إضافي بسبب branch interlock، كما أن خانة الفرع المؤجل لا تُستغل وتبقى كتعليمة فارغة (stall).

بسبب هذه التوقفات وعدم استغلال ال delayed branch slot، يحتاج التكرار الواحد من الحلقة غير المجدولة إلى 10 دورات ساعة.

Scheduled Code – Reducing to 6 Clock Cycles

Loop Example with Delayed Branch Slot

38

● **Scheduled code:** we need **6 cycles**

```

1  Loop: L.D F0,0(R1)
2          DADDUI R1,R1,#-8
3          ADD.D F4,F0,F2
4          stall                    #data hazard (F4)
5          BNE R1,R2,Loop
6          S.D F4,8(R1)            # 0+8 = 8

```

✎ في هذا المثال :

تم جدولة التعليمات بشكل أفضل لتقليل عدد دورات الساعة إلى 6 دورات فقط لكل تكرار من الحلقة.
 تم وضع تعليمة DADDUI بعد Load مباشرة، لأنها مستقلة ويمكن تنفيذها دون انتظار ووضعنا تعليمة ال S.D بعد تعليمة ال BNE
 بعد ذلك تأتي ADD.D، ولكن بسبب data hazard على الناتج F4، تحتاج إلى دورة انتظار واحدة.
 أخيرًا يتم تنفيذ تعليمة Store و BNE .

ملاحظة مهمة:

المؤشر F4 ينتج من ADD.D ويخزن في Store بعد دورة انتظار واحدة، لذا الفرق بين العنوان الأصلي والصحيح هو $8=8+0$ بايت لكل عنصر.
 بهذه الطريقة، استطعنا تقليل عدد دورات الساعة لكل تكرار من 9 دورات في الكود غير المجدول، إلى 6 دورات فقط.

Loop Unrolling Example

(الان نريد أن ن فك الحلقة)

في Loop Unrolling نقوم بتكرار جسم الحلقة عدة مرات داخل نفس التكرار، مع تعديل كود إنهاء الحلقة بما يتناسب.

فوائد هذه الطريقة:

1. إزالة التعليمة الشرطية Branch تسمح بجدولة التعليمات من تكرارات مختلفة معاً، مما يلغي حالات توقف البيانات (data stalls) .
2. زيادة نسبة العمل المفيد مقارنة بالتحكم overhead داخل الحلقة، أي أن المعالج يقضي وقت أكثر في تنفيذ العمليات الفعلية بدل الانتظار أو القفزات.
3. العيب الرئيسي هو الحاجة إلى عدد أكبر من السجلات لتخزين النتائج المؤقتة لكل تكرار مكرر

LOOP Unrolling Example – Step 1

LOOP Unrolling Example

40

- **First Step: Copy the Loop**
 - Assuming R1 – R2 (that is, the size of the array) is initially a multiple of 32, which means that the number of loop iterations is a multiple of 4.
- ```

1 Loop: L.D F0,o(R1)
2 ADD.D F4,F0,F2
3 S.D F4,o(R1) #drop DADDUI & BNE
4 L.D F0,-8(R1)
5 ADD.D F4,F0,F2
6 S.D F4,-8(R1) #drop DADDUI & BNE
7 L.D F0,-16(R1)
8 ADD.D F4,F0,F2
9 S.D F4,-16(R1) #drop DADDUI & BNE
10 L.D F0,-24(R1)
11 ADD.D F4,F0,F2
12 S.D F4,-24(R1)
13 DADDUI R1, R1, #-32 #-8 * 4 = 32
14 BNE R1,R2,LOOP
15 NOP

```

أول خطوة في Loop Unrolling هي نسخ الحلقة عدة مرات داخل نفس التكرار لتقليل عدد التعليمات المتكررة وحالات التوقف في خط الأنابيب.

## ❧ في هذا المثال :

نفترض أن طول المصفوفة R1-R2 هو من مضاعفات 32، أي أن عدد تكرارات الحلقة مضاعف لـ 4. نقوم بتحميل أربعة عناصر متتالية من المصفوفة ومعالجتها مباشرة ضمن نفس التكرار، بدل تنفيذ كل عنصر على حدة في تكرار مستقل.

## ■ التسلسل كالتالي:

- تحميل العنصر الأول ← L.D F0,0(R1) إضافة الثابت ← ADD.D F4,F0,F2 تخزين النتيجة S.D F4,0(R1)
- تحميل العنصر الثاني ← L.D F0,-8(R1) إضافة ← تخزين
- تحميل العنصر الثالث ← L.D F0,-16(R1) إضافة ← تخزين
- تحميل العنصر الرابع ← L.D F0,-24(R1) إضافة ← تخزين

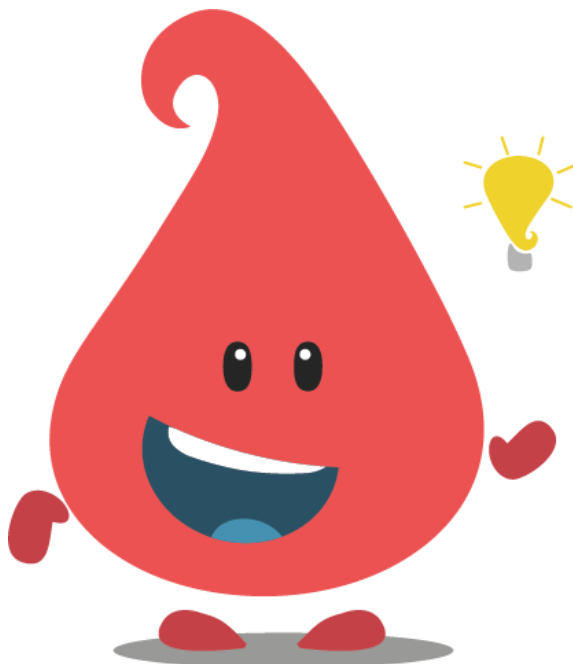
بعد معالجة الأربعة عناصر، نقوم بتحديث المؤشر R1 بمقدار 32  
`DADDUI R1,R1,# - 32`

ثم نقارن للانتقال للتكرار التالي `BNE R1,R2,LOOP`.

## ■ بهذه الطريقة:

- قللنا عدد القفزات والـ branch instructions
- استغللنا خط الأنابيب بشكل أفضل
- قللنا حالات stall و زدنا الأداء بشكل ملحوظ.

ملاحظة: كل أربعة عناصر تحتاج 32 بايت (4 × 8) بايت لكل (Double Word).



## Loop Unrolling Example – Step 2: Find Name Dependencies

## LOOP Unrolling Example

41

## ● Second Step: Find Name Dependencies

|    |                                  |                |
|----|----------------------------------|----------------|
| 1  | Loop: L.D F0,0(R1)               | #1st iteration |
| 2  | ADD.D F4,F0,F2                   |                |
| 3  | S.D F4,0(R1)                     |                |
| 4  | L.D <b>F0</b> , -8(R1)           | #2nd iteration |
| 5  | ADD.D <b>F4</b> , <b>F0</b> , F2 |                |
| 6  | S.D <b>F4</b> , -8(R1)           |                |
| 7  | L.D <b>F0</b> , -16(R1)          | #3ed iteration |
| 8  | ADD.D <b>F4</b> , <b>F0</b> , F2 |                |
| 9  | S.D <b>F4</b> , -16(R1)          |                |
| 10 | L.D <b>F0</b> , -24(R1)          | #4th iteration |
| 11 | ADD.D <b>F4</b> , <b>F0</b> , F2 |                |
| 12 | S.D <b>F4</b> , -24(R1)          |                |
| 13 | DADDUI R1, R1, #-32              |                |
| 14 | BNE R1,R2,LOOP                   |                |
| 15 | NOP                              |                |

في هذه الخطوة، نقوم بتطبيق Loop Unrolling على الحلقة لتقليل عدد القفزات واستغلال خط الأنابيب بشكل أفضل.

الحلقة الأصلية كانت تقوم بإضافة قيمة ثابتة s لكل عنصر من عناصر المصفوفة عنصرًا عنصرًا، مع تحديث المؤشر بعد كل عملية.

بعد Loop Unrolling، نقوم بعمل 4 تكرارات من التعليمات داخل نفس الحلقة:

نحمل العنصر الأول، نجمعه مع s، ونخزنه.

نحمل العنصر الثاني (offset -8)، نجمعه مع s، ونخزنه.

نحمل العنصر الثالث (offset -16)، نجمعه مع s، ونخزنه.

نحمل العنصر الرابع (offset -24)، نجمعه مع s، ونخزنه.

بعد ذلك، يتم تحديث المؤشر R1 بمقدار -32 للانتقال إلى العناصر التالية، ثم يتم تنفيذ تعليمة الفرع BNE للعودة إلى بداية الحلقة إذا لم تنتهِ جميع العناصر.

باستخدام Loop Unrolling :

نقل عدد مرات تنفيذ تعليمات BNE و DADDUI .

نستغل خط الأنابيب بشكل أفضل، حيث يمكن تنفيذ عدة عمليات مستقلة متداخلة دون توقف.

نحافظ على البيانات مستقلة لكل تكرار لتجنب أي Name Dependencies أو stalls .

## Loop Unrolling Example – Third Step: Register Renaming

## ● Third Step: Register Renaming

|                                       |                  |
|---------------------------------------|------------------|
| 1 Loop: L.D F0,0(R1)                  |                  |
| 2 ADD.D F4,F0,F2                      |                  |
| 3 S.D F4,0(R1)                        |                  |
| 4 L.D <b>F6</b> , -8(R1)              | # Renamed F0     |
| 5 ADD.D <b>F8</b> , <b>F6</b> , F2    | # Renamed F4, F0 |
| 6 S.D <b>F8</b> , -8(R1)              | # Renamed F4     |
| 7 L.D <b>F10</b> , -16(R1)            | # Renamed F0     |
| 8 ADD.D <b>F12</b> , <b>F10</b> , F2  | # Renamed F4, F0 |
| 9 S.D <b>F12</b> , -16(R1)            | # Renamed F4     |
| 10 L.D <b>F14</b> , -24(R1)           | # Renamed F0     |
| 11 ADD.D <b>F16</b> , <b>F14</b> , F2 | # Renamed F4, F0 |
| 12 S.D <b>F16</b> , -24(R1)           | # Renamed F4     |
| 13 DADDUI R1, R1, #-32                |                  |
| 14 BNE R1, R2, LOOP                   |                  |
| 15 NOP                                |                  |

## ✎ في هذا المثال :

تم تطبيق توسيع الحلقة (Loop Unrolling) مع إعادة تسمية السجلات (Register Renaming) لتحسين أداء خط الأنابيب:

- تم فك الحلقة لتعمل على أربعة عناصر في كل تكرار بدل عنصر واحد.
- هذا يقلل من عدد تكرارات الحلقة ويزيد التوازي بين التعليمات.
- تم إعادة تسمية السجلات القائمة لتجنب التعارضات بين نتائج التعليمات المختلفة.

## ✎ على سبيل المثال :

بعد استخدام F0 للعنصر الأول، يتم تحميل العنصر التالي في F6 بدل F0 ، وهكذا ...  
 التعليمات الأساسية لكل عنصر تبقى نفسها Load → Add → Store ؛ لكن مع أسماء سجلات مختلفة لكل خطوة لضمان عدم الاعتماد على نتائج سابقة.  
 بعد معالجة العناصر الأربعة، يتم تحديث المؤشر R1 بمقدار 32 بايت (4 عناصر × 8 بايت لكل عنصر)، ثم تنفيذ تعليمة الفرع BNE لاستكمال الحلقة.

## الفائدة:

زيادة عدد التعليمات المستقلة في خط الأنابيب، مما يقلل من حالات التوقف (stalls).  
 تحسين الأداء بشكل ملحوظ مقارنة بالحلقات غير الموسعة أو المجدولة بشكل بسيط.

## Loop Unrolling Example: Unrolled, Unscheduled

## LOOP Unrolling Example: Unrolled, Unscheduled

43

Loop: L.D F0, 0(R1)

1Stall

ADD.D F4, F0, F2

2 Stalls

S.D F4, 0(R1) ;

L.D F6, -8(R1)

1Stall

ADD.D F8, F6, F2

2 Stalls

S.D F8, -8(R1) ;

L.D F10, -16(R1)

1Stall

ADD.D F12, F10, F2

2 Stalls

S.D F12, -16(R1) ;

L.D F14, -24(R1)

1Stall

ADD.D F16, F14, F2

2 Stalls

S.D F16, -24(R1)

DADDUI R1, R1, #-32

1 Stalls

BNE R1, R2, Loop

1 Stalls

Total of 28 clock cycles for every 4 elements in the loop OR 7 cycles per iteration!

في هذا المثال، تم توسيع الحلقة (Loop Unrolling) بحيث يتم معالجة أربعة عناصر من المصفوفة في كل تكرار بدلاً من عنصر واحد فقط.

تعليمات الحلقة الآن تحتوي على أربع عمليات Load → Add → Store متتابة لكل عنصر،

ثم تحديث المؤشر R1 وأخيراً تعليمة الفرع BNE .

حتى مع توسعة الحلقة، وبما أن الكود غير مجدول (Unscheduled)، تظهر عدة stalls بين التعليمات بسبب الاعتماد على نواتج التحميل والجمع.

في هذا المثال، هناك :

- 4 stalls بعد بعض التعليمات
- 2 stalls في أماكن أخرى
- 1 stall أحياناً قبل الفرع

النتيجة:

كل تكرار من الحلقة يعالج 4 عناصر ويستغرق 28 دورة ساعة. أي أن كل عنصر يُعالج في المتوسط 7 دورات ساعة، مثل الحلقة المجدولة الأساسية بدون التوسيع، لكننا حصلنا على أربعة عناصر في تكرار واحد مما يقلل عدد مرات تكرار الحلقة ويقلل التحكم بالفرع.

## Loop Unrolling Example: Unrolled and Scheduled

### After Scheduling the Unrolled Loop

```

1 Loop: L.D Fo, o(R1)
2 L.D F6, -8(R1)
3 L.D F10, -16(R1)
4 L.D F14, -24(R1)
5 ADD.D F4, Fo, F2
6 ADD.D F8, F6, F2
7 ADD.D F12, F10, F2
8 ADD.D F16, F14, F2
9 S.D F4, o(R1)
10 S.D F8, -8(R1)
11 DADDUI R1, R1, #-32
12 S.D F12, 16(R1) # -16+32 = 16
13 BNE R1, R2, Loop
14 S.D F16, 8(R1) # -24+32 = 8

```

**In total only 14 Clock cycles for 4 elements!! or 3.5 cycles per iteration!**

- The gain from scheduling on the unrolled loop is even larger than on the original loop!!

في هذا المثال تم تطبيق فك الحلقة (Loop Unrolling) بمقدار أربع مرات، ثم تمت جدولة التعليمات لتقليل حالات التوقف في خط الأنابيب.

يتم أولاً تحميل أربعة عناصر متتالية من المصفوفة باستخدام أربع تعليمات Load مستقلة. بعد ذلك، تُنفَّذ أربع عمليات جمع ADD.D لكل عنصر مع القيمة الثابتة s، مستفيدين من تداخل التنفيذ داخل خط الأنابيب.

ثم يتم تخزين النتائج مرة أخرى في الذاكرة باستخدام تعليمات Store، مع تحديث المؤشر R1 مرة واحدة فقط بمقدار 32 بايت بدل تحديثه في كل تكرار.

تم تعديل عناوين بعض تعليمات التخزين لتعويض تغيير قيمة R1 بعد التحديث.

بفضل فك الحلقة والجدولة، تم تنفيذ 4 عناصر خلال 14 دورة ساعة فقط، أي بمعدل 3.5 دورة لكل عنصر. وهذا يوضح أن تحسين الأداء الناتج عن الجدولة مع فك الحلقة أكبر بكثير من التحسين الناتج عن الجدولة وحدها في الحلقة الأصلية.

على الرغم من أن Loop Unrolling يحسّن الأداء، إلا أن له بعض القيود:

- أولاً: انخفاض مقدار الفائدة من تقليل الحمل الزائد (overhead) مع كل زيادة في درجة فك الحلقة، لأن جزء التحكم يصبح أصغر تأثيراً نسبياً.
- ثانياً: قيود حجم الشيفرة، حيث يؤدي فك الحلقة إلى زيادة حجم البرنامج، مما قد يؤثر على الذاكرة وكفاءة جلب التعليمات.
- ثالثاً: قيود المترجم (Compiler Limitations)، إذ قد يؤدي الفك العدواني للحلقات مع الجدولة المكثفة إلى نقص في عدد السجلات المتاحة، وهو ما يُعرف بـ Register Pressure، مما يحد من إمكانية التحسين.

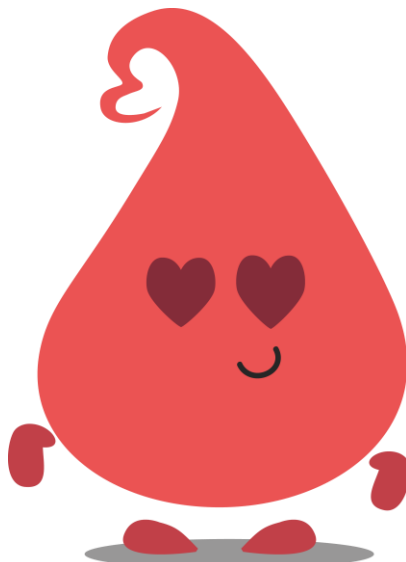
## Compiler Perspectives: Unrolling Loops

من منظور المترجم، غالباً لا يكون الحد الأعلى لعدد تكرارات الحلقة معروفاً أثناء الترجمة. لنفترض أن عدد التكرارات هو  $n$ ، ونريد تنفيذ  $k$  نسخ من جسم الحلقة باستخدام loop unrolling.

في هذه الحالة، لا يقوم المترجم بإنشاء حلقة واحدة فقط بعد فكها، بل يُنشئ حلقتين متتاليتين:

- الحلقة الأولى تنفذ الجسم الأصلي للحلقة عدد  $(n \bmod k)$  مرات، وذلك لمعالجة التكرارات المتبقية.
- الحلقة الثانية تنفذ جسم الحلقة بعد فكّه عدد  $(n / k)$  مرات.

وبما أن قيمة  $n$  تكون كبيرة في العادة، فإن معظم التكرارات يتم تنفيذها داخل الحلقة المفكوكة، مما يقلل عدد القفزات ويحسن الأداء.



## Compiler Perspectives: Dependencies

يجب على المترجم (Compiler) المحافظة على اعتماديات البيانات (Data Dependencies) عند إعادة ترتيب أو تحسين الشيفرة، لأن أي كسر لهذه الاعتماديات يؤدي إلى نتائج غير صحيحة.

- أولاً، يحدد المترجم ما إذا كانت تكرارات الحلقة مستقلة عن بعضها، حتى يتمكن من تطبيق تقنيات مثل loop unrolling أو instruction scheduling بأمان.
- ثانياً، أثناء عملية unrolling، يقوم المترجم بإعادة تسمية السجلات لتجنب التعارض بين القيم الناتجة من تكرارات مختلفة.
- ثالثاً، يعمل المترجم على تقليل أو إزالة تعليمات الاختبار والقفز غير الضرورية، مع تعديل آلية تحديث الحلقة وإنهائها بما يتناسب مع الترتيب الجديد.
- كما يحدد ما إذا كان من الممكن تبديل تعليمات التحميل والتخزين دون التأثير على صحة البيانات.
- أخيراً، يقوم المترجم بجدولة التعليمات بطريقة تحسّن الأداء، مع الحفاظ الكامل على جميع الاعتماديات.

## Compiler Perspectives: Renaming

التعليمات المعتمدة على بعضها لا يمكن تنفيذها بالتوازي.

بالنسبة للسجلات، يكون تحديد الاعتماد سهلاً لأن أسماء السجلات ثابتة وواضحة. أما في الذاكرة، فالأمر أصعب بكثير، وتُعرف هذه المشكلة باسم Memory Disambiguation، أي تحديد ما إذا كانت تعليمتان تتعاملان مع نفس موقع الذاكرة أم لا.

### ⚡ على سبيل المثال :

لا يمكن الجزم بسهولة ما إذا كان العنوان  $100(R4)$  يساوي  $20(R6)$ ، أو ما إذا كان الوصول إلى نفس العنوان يتم في تكرارات مختلفة من الحلقة.

في مثالنا، يجب على المترجم أن يحدد أنه إذا كان السجل  $R1$  لا يتغير إلا بقيمة ثابتة، فإن العناوين:

$0(R1)$ ، و  $8-(R1)$ ، و  $16-(R1)$ ، و  $24-(R1)$

تشير إلى مواقع مختلفة في الذاكرة.

بناءً على ذلك، يمكن للمترجم اعتبار تعليمات load و store مستقلة، وبالتالي يمكن إعادة ترتيبها أو تبديلها لتحسين الأداء دون التأثير على صحة البرنامج.

## Introduction to Cache Memory

Cache Memory هي ذاكرة صغيرة و قريبة من المعالج وسريعة جداً تقع بالقرب من المعالج (Processor) .

1. لماذا هي أسرع من RAM ؟

- الـرام (RAM – Random Access Memory) مصنوعة من transistors أو switches عادية.
- الكاش مؤلفة من Flip-Flops، وكل Flip-Flop يحتوي على الأقل 8 transistors ←
- لذلك، كل خلية في الكاش  $\approx 8$  أضعاف تكلفة خلية في الـرام.
- الكاش أعلى ولكن أسرع بكثير.

2. مستويات الذاكرة: (Memory Hierarchy)

كلما اقتربت الذاكرة من المعالج: أسرع وأعلى وأصغر حجماً.  
كلما ابتعدت الذاكرة عن المعالج: أبطأ و أرخص و أكبر حجماً.

3. مبدأ الـ Locality (مبدأ المحلية):

مبدأ الـ Locality أو مبدأ المحلية هو مفهوم أساسي في تنظيم الذاكرة في الحاسوب.  
الفكرة الرئيسية منه هي أن البرنامج أثناء التنفيذ لا يستخدم كل الذاكرة مرة واحدة، بل يستخدم جزءاً صغيراً منها فقط خلال فترة زمنية معينة.

ينقسم مبدأ الـ Locality إلى نوعين رئيسيين:

1. أولاً Temporal Locality (المحلية الزمنية) :

وهي تعني أن البرنامج إذا استخدم بيانات أو تعليمة معينة الآن، فمن المحتمل جداً أن يستخدمها مرة أخرى قريباً في الزمن.

مثال: عند استخدام متغير داخل حلقة تكرار، سيتم استخدام هذا المتغير مرات عديدة خلال وقت قصير.  
لهذا السبب تعتمد ذاكرة الكاش (Cache Memory) على هذا النوع من المحلية، حيث تقوم بتخزين البيانات التي تم استخدامها مؤخراً، لأن احتمال استخدامها مرة أخرى قريباً يكون كبيراً، وهذا يؤدي إلى تسريع تنفيذ البرنامج.

## 2. ثانياً Spatial Locality (المحلية المكانية) :

وهي تعني أنه إذا قام البرنامج بالوصول إلى موقع معين في الذاكرة، فمن المحتمل أن يصل إلى مواقع قريبة منه في الذاكرة أيضاً.

مثال: عند قراءة عناصر مصفوفة، غالباً يتم قراءة العناصر المتجاورة في الذاكرة بشكل متسلسل. بسبب هذا النوع من المحلية، تقوم ذاكرة الكاش بتخزين مجموعة من الكلمات المتجاورة في الذاكرة وليس كلمة واحدة فقط.

كلما كان حجم البلوك (Block Size) أكبر من كلمة واحدة، زادت الاستفادة من الـ Spatial Locality ، وبالتالي تتحسن سرعة الوصول للبيانات.

## الخلاصة:

مبدأ Locality يفسر لماذا تعمل ذاكرة الكاش بكفاءة عالية، لأن البرامج عادة تستخدم نفس البيانات بشكل متكرر (Temporal) وتستخدم بيانات متجاورة في الذاكرة (Spatial) ، وهذا يساعد على تسريع أداء النظام بشكل كبير.

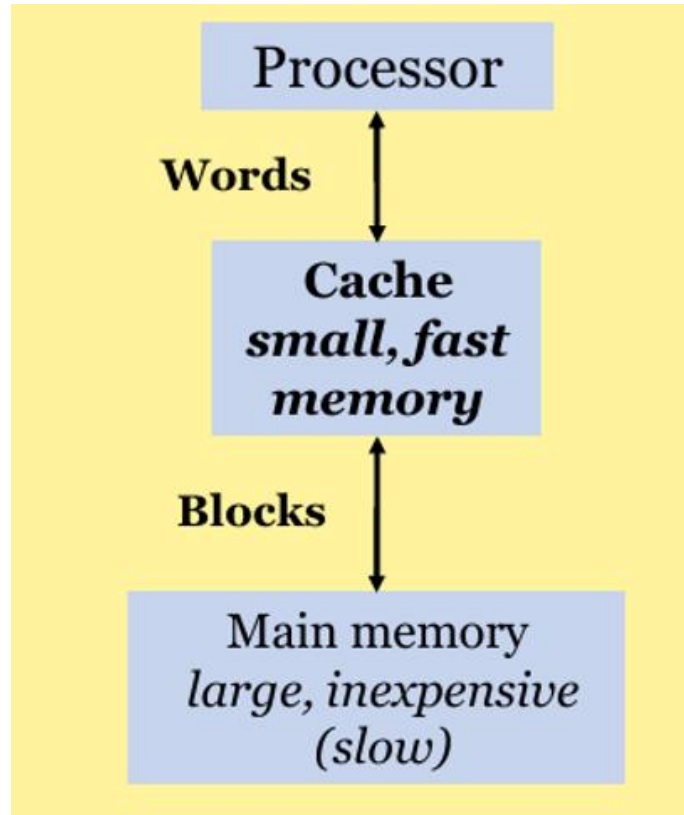
## How Cache Works آلية عمل الكاش

- Processor يولد العنوان النهائي (Physical Address)
- الكاش تجلب Block كامل من البيانات، ليس كلمة واحدة فقط.
- Word وحدة بيانات المعالج (Processor Word) تختلف حسب المعالج (Integer, Float)
- العنوان النهائي Address يولده المعالج.

الـ Cache Memory هي ذاكرة صغيرة وسريعة جداً تقع بين المعالج (Processor) والذاكرة الرئيسية (Main Memory). هدفها الأساسي هو تسريع تنفيذ البرامج وتقليل الوقت الذي يحتاجه المعالج للوصول إلى البيانات. المعالج لا يتعامل مباشرة مع الذاكرة الرئيسية في كل مرة، بل يقوم بجميع عمليات القراءة والكتابة أولاً مع الـ Cache وذلك لأن الـ Cache أسرع بكثير من الذاكرة الرئيسية

## لدينا تسلسل العمل التالي:

المعالج يطلب البيانات ← يبحث أولاً في الـ Cache ← إذا لم يجدها ينتقل إلى الذاكرة الرئيسية.



## 1. أولاً : مفهوم Hit :

عندما يطلب المعالج بيانات ويجدها موجودة داخل الـ Cache ، تسمى هذه الحالة Hit .  
في هذه الحالة تتم عملية القراءة أو الكتابة مباشرة من الـ Cache دون الرجوع إلى الذاكرة الرئيسية، وبالتالي تكون العملية سريعة جداً.

## 2. ثانياً: مفهوم Miss :

إذا طلب المعالج بيانات ولم يجدها في الـ Cache ، تسمى هذه الحالة Miss .  
عند حدوث Miss يتم جلب البيانات من الذاكرة الرئيسية إلى الـ Cache أولاً، وليس فقط الكلمة المطلوبة بل يتم جلب بلوك كامل يحتوي عليها، وبعدها يتم تنفيذ طلب المعالج.

## 3. ثالثاً: مفهوم Block :

الـ Block هو أصغر كمية بيانات يتم نقلها بين الذاكرة الرئيسية والـ Cache .  
بدل نقل كلمة واحدة فقط، يتم نقل مجموعة كلمات متجاورة، وذلك للاستفادة من مبدأ الـ Spatial Locality وتحسين السرعة.

لماذا نستخدم Cache Memory ؟

لأن الذاكرة الرئيسية كبيرة ورخيصة لكنها بطيئة، بينما الـ Cache صغيرة لكنها سريعة جداً.  
لذلك استخدام الـ Cache يقلل زمن الوصول للبيانات ويحسن أداء النظام بشكل كبير.

## Cache Performance

- Average Access Time (ACT)
 
$$= T_1 \times h + (T_m + T_1) \times (1 - h)$$

$$= T_1 + T_m \times (1 - h)$$
 where
  - $T_1$  = cache access time (small)
  - $T_m$  = memory access time (large)
  - $h$  = hit rate ( $0 \leq h \leq 1$ )
- Hit rate is also known as hit ratio,  
**miss rate = 1 – hit rate**

متوسط زمن الوصول إلى الذاكرة (Average Access Time (ACT) ، هو الزمن الوسطي الذي يحتاجه المعالج للوصول إلى البيانات عند استخدام الـ Cache والذاكرة الرئيسية معاً.

بما أن المعالج يبحث أولاً في الـ Cache ، فقد يجد البيانات مباشرة (Hit) ، أو قد لا يجدها وينتقل إلى الذاكرة الرئيسية (Miss) ، لذلك نحسب الزمن المتوسط للوصول باستخدام معادلة خاصة.  
 معادلة متوسط زمن الوصول هي:

$$ACT = T_1 \times h + (T_m + T_1) \times (1 - h)$$

ويمكن تبسيطها إلى:

$$ACT = T_1 + T_m \times (1 - h)$$

الرموز في المعادلة:

- $T_1$ : هو زمن الوصول إلى الـ Cache ، وهو صغير جداً لأن الـ Cache سريعة.
- $T_m$ : هو زمن الوصول إلى الذاكرة الرئيسية، وهو أكبر بكثير لأنها أبطأ
- $h$ : هو معدل النجاح (Hit Rate) ، أي نسبة المرات التي يجد فيها المعالج البيانات داخل الـ Cache ، وقيمتها بين 0 و 1.

1. عند وجود Hit :

المعالج يحصل على البيانات مباشرة من الـ Cache بزمن  $T_1$  فقط.

2. عند حدوث Miss :

المعالج يبحث في الـ Cache أولاً ( $T_1$ ) ثم يذهب إلى الذاكرة الرئيسية ويستغرق زمن إضافي  $T_m$  ، لذلك الزمن الكلي في حالة Miss هو  $(T_1 + T_m)$  .

لذلك نحسب المتوسط اعتماداً على احتمال حدوث Hit واحتمال حدوث Miss

3. معدل Hit و Miss :

Hit Rate : هو نسبة المرات التي يتم فيها العثور على البيانات داخل الـ Cache .

Miss Rate : هو نسبة المرات التي لا يتم العثور فيها على البيانات داخل الـ Cache .

العلاقة بينهما:

$$\text{Miss Rate} = 1 - \text{Hit Rate}$$

الخلاصة:

كلما زاد Hit Rate واقترب من 1، قل متوسط زمن الوصول إلى الذاكرة، وبالتالي يتحسن أداء النظام بشكل كبير. أما إذا كان Hit Rate منخفضاً، سيضطر المعالج للذهاب إلى الذاكرة الرئيسية كثيراً، مما يزيد زمن التنفيذ ويبطئ الأداء.

مثال :

- Ideal processor with 1 cycle memory access, CPI = 1
- Processor without cache:
  - ✦ Assume main memory access time of **10 cycles**
  - ✦ Assume 30% instructions require memory data access
- Processor with cache:
  - ✦ Assume cache access time of 1 cycle
  - ✦ Assume **hit rate 0.95** for instructions, **0.90 for data**
  - ✦ Assume miss penalty (time to read memory into cache and from cache to processor) is **11 cycles**

○ Comparing times of 100 instructions:

$$\frac{\text{Time without cache}}{\text{Time with cache}} = \frac{100 \times 10 + 30 \times 10}{100(0.95 \times 1 + 0.05 \times 11) + 30(0.9 \times 1 + 0.1 \times 11)}$$

$$= 6.19$$

شرح المثال :

في هذا المثال نقارن بين أداء المعالج في حالتين:

- معالج بدون Cache  
- معالج يستخدم Cache

وذلك لمعرفة كم تحسن الأداء عند استخدام ذاكرة الكاش.

نفترض أولاً أن لدينا معالج مثالي (Ideal Processor) يحتاج دورة واحدة فقط لتنفيذ التعليمة، أي أن:  $CPI = 1$  ، لكن المشكلة الحقيقية تكون في زمن الوصول إلى الذاكرة.

أولاً: المعالج بدون Cache

المعطيات:

- زمن الوصول إلى الذاكرة الرئيسية = 10 cycles
- 30% من التعليمات تحتاج للوصول إلى بيانات من الذاكرة

نريد حساب الزمن لتنفيذ 100 تعليمة

كل تعليمة تحتاج للوصول إلى الذاكرة لجلب التعليمة نفسها، أي:

$$100 \text{ تعليمة} \times 10 \text{ cycles} = 1000 \text{ cycles}$$

ولدينا 30 تعليمة إضافية (30%) تحتاج للوصول للبيانات من الذاكرة:

$$30 \times 10 = 300 \text{ cycles}$$

إذن الزمن الكلي بدون Cache:

$$1000 + 300 = 1300 \text{ cycles}$$

ثانياً: المعالج مع Cache

المعطيات:

- زمن الوصول للـ Cache = 1 cycle
- Hit rate للتعليمات = 0.95
- Hit rate للبيانات = 0.90
- Miss penalty = 11 cycles (أي زمن عدم وجود البيانات في الكاش و الذهاب للذاكرة)

نحسب الزمن لـ 100 تعليمة:

1. أولاً: زمن جلب التعليمات

احتمال Hit = 0.95 ← الزمن = 1 cycle

احتمال Miss = 0.05 ← الزمن = 11 cycles

الزمن الوسطي للتعليمة الواحدة:

$$(0.95 \times 1) + (0.05 \times 11)$$

لكل 100 تعليمة:

$$100 \times (0.95 \times 1 + 0.05 \times 11)$$

2. ثانياً: زمن جلب البيانات (30 تعليمة تحتاج بيانات للذهاب للرام)

Hit rate = 0.9

Miss rate = 0.1

الزمن الوسطي لكل عملية بيانات:

$$(0.9 \times 1) + (0.1 \times 11)$$

لكل 30 عملية:

$$30 \times (0.9 \times 1 + 0.1 \times 11)$$

3. الحساب النهائي:

زمن التنفيذ مع الكاش =

$$100(0.95 \times 1 + 0.05 \times 11) + 30(0.9 \times 1 + 0.1 \times 11)$$

$$= 100(0.95 + 0.55) + 30(0.9 + 1.1)$$

$$= 100(1.5) + 30(2)$$

$$= 150 + 60$$

$$= 210 \text{ cycles تقريباً}$$

لكن بالمقارنة النسبية مع النظام القديم نحصل على:

$$\frac{\text{Time without cache}}{\text{Time with cache}} = 6.19 \text{ مرات أسرع}$$

*The end*